SIH 2026 — Internal Practical Assessment

Yashwanth N V · Reg 411723104058 · PSVPEC · CSE PSVPEC · Year IV

Level: Easy	**Duration:** 2 days	**Marks:** 70

Problem Statement — College Placement Drive and Student Application Tracker

Problem Description

A placement cell circulates company notices by message and collects applications on paper. Which student applied to which company, who was shortlisted, and who finally received an offer is spread across separate lists that are never reconciled. The cell therefore cannot answer basic questions about its own performance, and students miss deadlines they never saw.

Objective

To build a tracker that records each drive, the students who applied, and the stage each application has reached, and shows the placement officer the current status of every drive and each student's outcome.

Tools You May Use

HTML/CSS/JS or React for the screens with Python (Flask/Django) or Node (Express) and SQLite, MySQL or PostgreSQL. Python with scikit-learn. Use a small standard classifier; nothing built from scratch. Docker and docker-compose with GitHub Actions — all free tiers.

How your declared roles are used in this project. This project is built from **Full Stack, AI/ML, Cloud/ DevOps**. Every task below belongs to one of these. You also selected Backend, Frontend; those are not required for this assessment and this paper is your complete project.

If you run short of time, work in this order. The first three tasks are the core and must be complete and working. The later build task may be kept small — say so in your README. A smaller, working solution scores better than several half-finished pieces.

Step-by-Step Tasks

Task 1: Prepare the Sample Data

Create a realistic dataset of about 100 placement application records with these fields: application_id, student_id, student_name, company, drive_date, stage, offer_status, package. Document what every field means and what values it may take. Include a few awkward cases on purpose — a missing value, two very similar names, and a record with nothing related to it — because you will need them when you test. Because your project includes a model, make sure the history is long enough to learn from and include a column recording the outcome you will later predict.

> **Logic:** Every later task reads from this one dataset, so its shape decides what you can build. The awkward cases are deliberate: they are what force you to decide what the screen shows when a value cannot be calculated, and they are what will test your search.

Task 2: Build the Register End to End

Build the screen where the placement coordinator records a placement application record and the backend that stores it. Validate every field on the server before saving, calculate any derived figure on the server, and show the result on the screen. One complete action must work from the screen through to stored data.

> **Logic:** A vertical slice that genuinely works end to end is worth more than two halves that each look finished. Validating and calculating on the server is what makes every user see the same number.

Task 3: Build the Prediction Model

Choose a target that is genuinely uncertain — which cases will need attention, which are at risk, which are likely to be delayed. Do NOT train a model to reproduce a fixed rule you could write with an if-statement. Using your seeded history, train one simple standard classifier, using only information available at the moment the prediction would be needed. Split into training and test sets before training and fix the random seed.

> **Logic:** The commonest mistake here is feeding the model the very information that reveals the answer, which produces an excellent score and a model that is useless because those fields do not exist yet when the prediction is required. Restricting the inputs is what separates a real prediction from a self-fulfilling one.

Task 4: Containerise the Application

Write a Dockerfile that installs dependencies before copying the source so a code change does not force a full reinstall, and keep every secret and environment-specific value out of the image entirely. Provide an example environment file containing placeholder values only.

> **Logic:** Anything placed inside an image is permanently visible to whoever obtains it, including in the layer history after it is removed. Supplying configuration at run time is also what lets the same image serve more than one environment.

Task 5: Integrate and Test the Whole Thing

Connect every part and test it properly: confirm the main flow works from start to finish with your own data; confirm the model's output appears where the decision is made and that a low-confidence case produces no forced prediction; check one calculated figure by hand against your data. Handle the loading, empty and error states so no screen or output is ever blank or fails silently.

> **Logic:** Each part can work perfectly alone while the whole fails at the joins, and this is the only test that proves otherwise. Checking a figure by hand is how you know the number the placement officer will act on is actually right.

Task 6: Document and Demonstrate

Write a README stating the problem in two lines, how to run your work step by step, what every field means, how any derived figure is calculated, and what is not finished. Take three or four screenshots of your own working solution, record a short demonstration video, and upload everything to one public GitHub repository.

> **Logic:** Somebody else must be able to run and understand this without you sitting beside them. Explaining how a derived figure is calculated matters most, because that is the number people will act on.

Please note: this is an Easy-level assessment. A simple, clear solution that works is exactly what is expected — advanced features are not required. Every task builds on the previous one, so complete them in order. All work is done on the computer; nothing has to be built or purchased. Submit everything through one public GitHub repository. If you cannot complete a task, submit what you have and state in one line what remains.